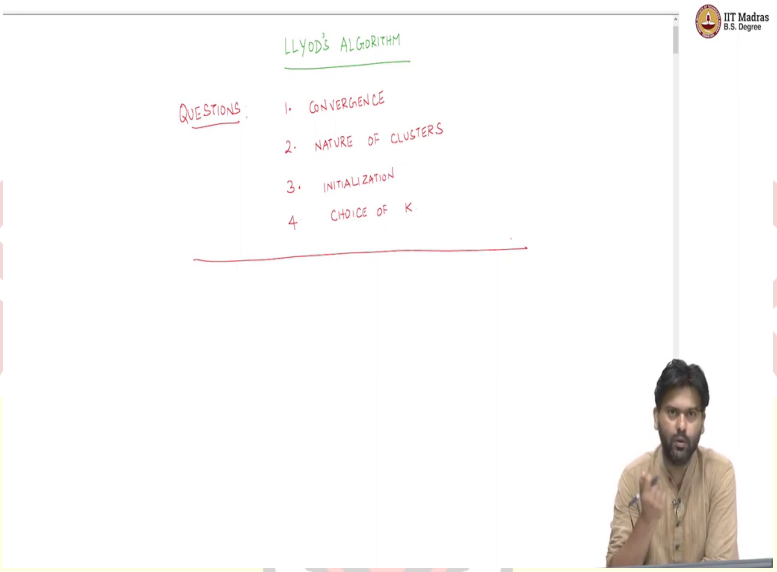
IIT Madras

ONLINE DEGREE

Machine Learning Techniques
Professor Arun Raj Kumar
Department of Computer Science and Engineering
Indian Institute of Technology Madras
Convergence of K-means algorithm

(Refer Slide Time: 00:13)



Hello, and welcome back, we are looking at the unsupervised learning specifically the problem of clustering. And last time we put down a specific algorithm called the Lloyd's Algorithm, or the k-means algorithm for this problem. And we try to understand intuitively what this algorithm does, the algorithm, if you remember, it is a simple algorithm, you start with an potentially arbitrary partition of the data points into clusters.

And then every point looks at the distance of the point to its own mean, and compares it with the mean of other clusters. And if it finds a cluster whose mean is closer to it in terms of distance squared, then the distance squared to its own mean, then it jumps to that cluster. And this happens for every point. And that is how a reassignment of points to partition or clusters happen. And we argued that we will do this until convergence.

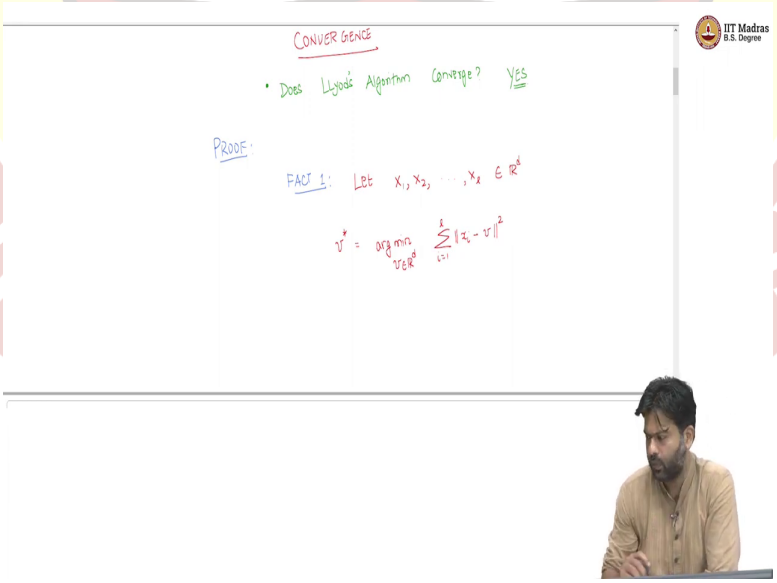
Now, what does convergence mean? Here is a question that we did not answer at that point. And that is one of the questions that comes up about the Lloyd's Algorithm which we need to answer. There are a couple of other questions also, for example, what kind of clusters does this algorithm produce? And what type of initialization should we do to get to reasonably good clusters?

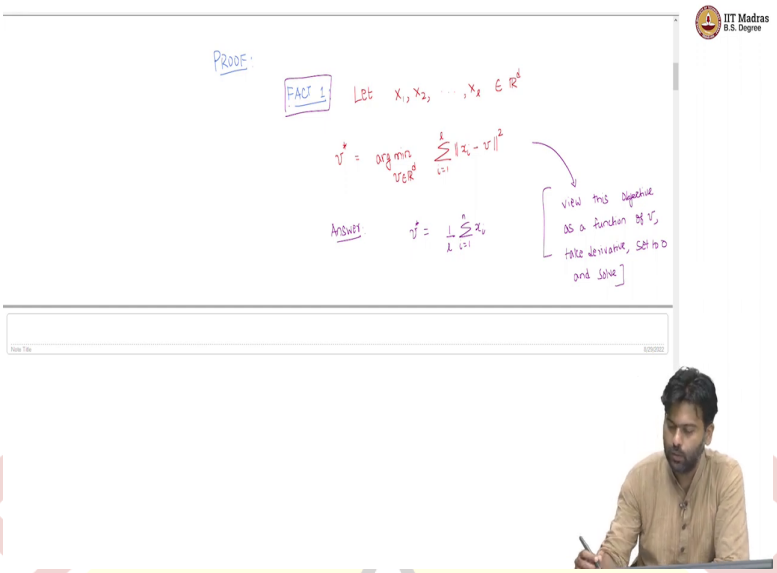
And finally, how do you choose K, because we are only given the data points, nobody tells us that there are K different partitions or clusters in this data. In some cases, it might be natural in the problem that you are trying to solve, in some cases, it may not be, for example, if you are trying to partition if you are a teacher who is trying to partition students into groups based on their performance in exams, and so on. And then you want to decide what grade to give to students.

Now, you know that there are only a set of grades S, A, B, C, D, for instance. And now you want to divide your students based on their marks in different exams into one of these buckets where you know the bucket size, or the number of grades on K, in the K-means algorithm. On the other hand, there might be cases where you do not know K, you are just given a bunch of points.

And then you want to figure out if there is any pattern where we don't know K, and then we need to come up with a way to figure out what is a good K. So, there are these four points, or these four questions that we will try to answer one by one as we go along in this course. So, the first question is the question of convergence. So, let us start with that question. Which is the question of convergence.

(Refer Slide Time: 02:57)





Does the Lloyd's algorithm converge? I did say last time that the algorithm does converge but we need to argue why. And we will do that right now. So, and understanding this argument also will tell us what kind of partitions that we will end up getting. Converge. So, the answer is yes. But then it needs an argument. So, to argue this, let us do the following. So, here is the proof. So, we will start with a simple fact.

Let us call this fact 1, which will be useful for our proof, because we are looking at data points and distance to other points and so on this proof is based on simple fact. Let us look at what the fact is. So, let us say you have a bunch of points, $x_1, x_2, \ldots, x_l \in R^d$, some set of points.

And now you want to find out, let us say, that point, which I am going to call as v^*, that minimizes over all possible vectors v in the dimension, the distance, the sum of the distance squared, i equals 1 to l, or the average of the distance squared does not really matter if it is sum or average, as we will see, to the bunch of points. This is a Euclidean distance squared.

And what we are asking is that you have a bunch of points and you want a v, which minimizes the sum of the distance squared, or the average of the district square average is just a scaling of the sum, so the minimizer will not change. We can ask either for the one that minimizes the sum or the average, in this case, let us say it we will look at the average. And we will not understand which one does this or even sum.

So, let us give it a sum, that is easier. Because we defined our partition function originally as sum. So, let us go on with the sum. So, what do you think this v would be? So, I am going to

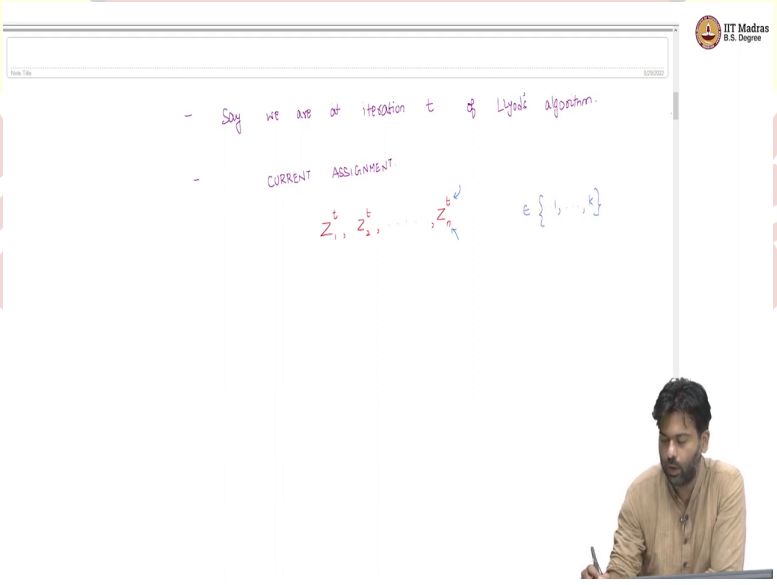
argue what this v^* which minimizes this would be, but then it might be a useful exercise to pause for a bit and think about what might be the answer to this problem. So, the answer to this is, v^* is just the mean of the data points.

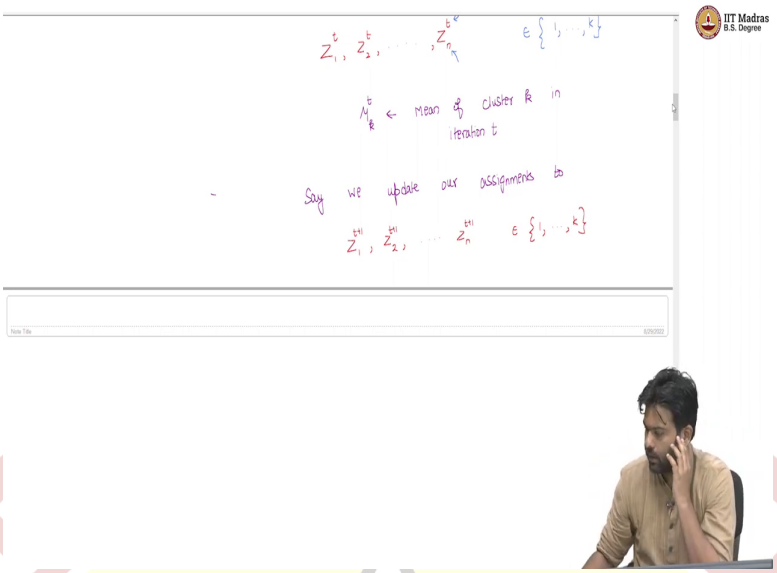
We have a bunch of data points, and then you are asking which point minimizes the Euclidean distance squared to all the data points, the sum of that is minimized by which point, well, that happens to be the mean of this bunch of data points. So, well, you can view this objective as a function of v, take derivative, set it to 0, and solve. So, that is how we would get this, I will not go through the steps. It is a very simple derivation.

And that is why I am not doing it. So, I would strongly encourage you to try this out. Especially if you are not so used to taking derivatives with respect to vectors. So, those would be like, not just a single value, the derivative would be like derivative in each direction for each component, so that would be like a gradient vector.

And then you want to treat this as a function of v, take the gradient with respect to v, set it to 0 and see which v solves that, and you would see that it would be the mean, that is a useful exercise to do. For us, it is just a fact, we will hold on to this fact, we will use this later on when we need it.

(Refer Slide Time: 07:11)





So, now we are going to prove why the Lloyd's algorithm actually converges in a very nice way. So, let us say we are at some iteration, let us call this some iteration t of the Lloyd's algorithm. Now, let us say our current assignment of points to clusters looks as follows. So, let us call the current assignment $Z_1^t, Z_2^t, ..., Z_n^t$, where of course the t corresponds to the iteration number, and the subscript corresponds to the particular data point. Remember, all of this are between 1 to k, each of these takes a value between 1 to k, because these are cluster indicators in our notation.

Now, let us also define μ_k^t as the mean of cluster k in iteration t. So, once you have a partition and the t-th term, there is some partition we are currently at, and each point has its own cluster indicator variable. So, you can basically the Zs will tell you how the data points go into buckets. And each bucket you can compute the mean. And let us call that mean as μ_k^t.

Now, what happens next is, well, the algorithm does a reassignment. We are trying to prove convergence of the algorithm. So, let us say at step t, we assume that the algorithm does not converge. Well, if the algorithm converges in step t, then there is nothing to prove. So, because you are attempting to prove that the algorithm converges, so let us assume the algorithm does not converge and see what happens.

So, there is some reassignment, which means at least one point is not happy with the distance squared to its own mean in its box, but then it has found a different box with a mean, whose

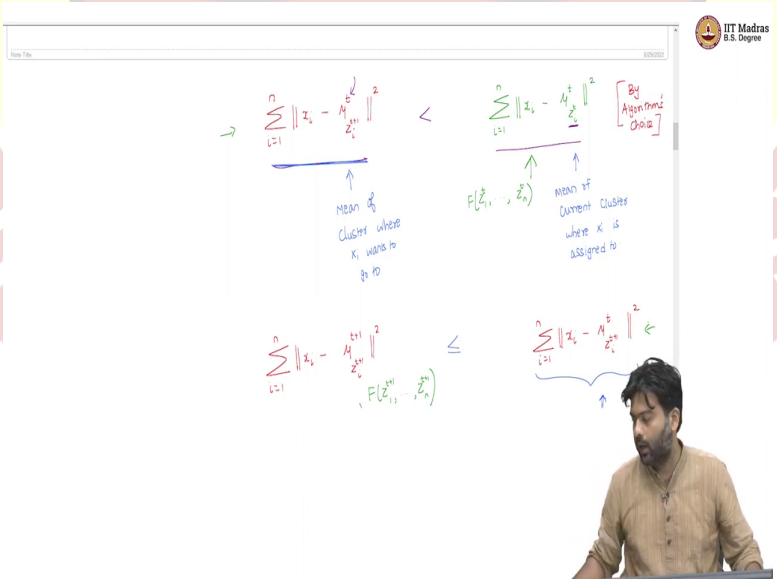
distance squared to the point is closer, strictly closer than the current mean. So, that is when a reassignment happens.

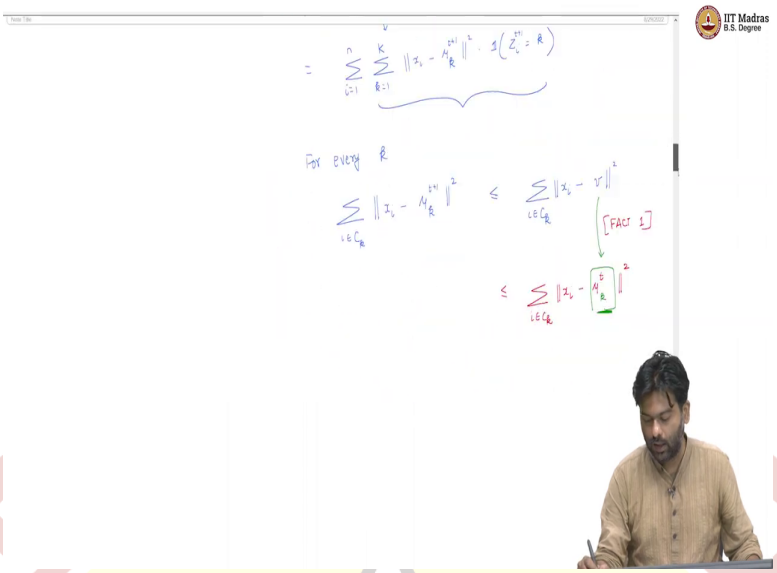
So, let us say we update our assignments to the next steps assignments. Now, these would be $Z_1^{t+1}, Z_2^{t+1}, ..., Z_n^{t+1} \in \{1, ..., k\}$. But then things have changed. So, points have moved around. The question is what can we say about this update? So, when we move from one partition, which is given by Z_1^t to Z_n^t to a different partition, which is given by Z_1^{t+1} to Z_n^{t+1}, what can we say? Is this a good change? That is what we are trying to capture.

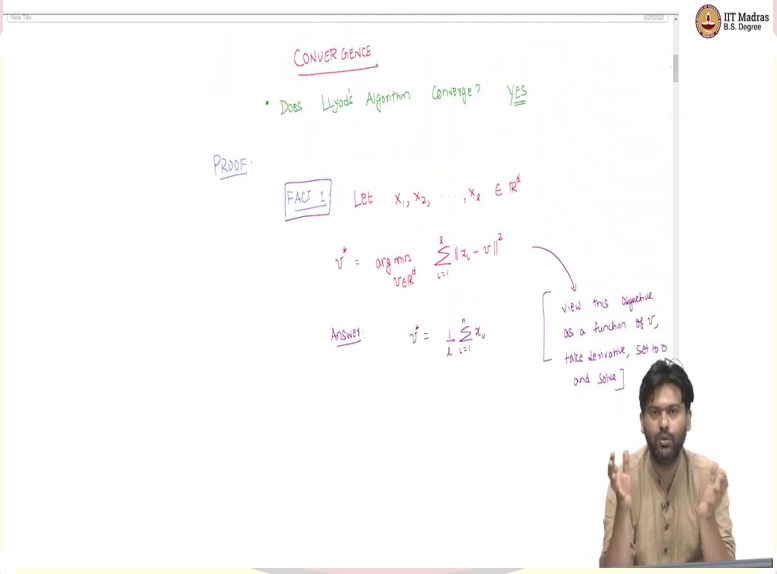
So, you are in some way of putting points in boxes. Now, you are moving points around. Now, after this the partition that results, is it a better partition than the previous partition? So, in some sense, that is what we are trying to understand. So, how do we argue this is a better partition?

Well, we have to look at our objective function, we had an objective function which given a partition will tell you how good that partition is by assigning it a number. And then we said that we wanted the smallest value for that objective function, the partition that gives us the smallest value. So, in terms of that, what can we say?

(Refer Slide Time: 11:06)







So, let us take a look at this. Let us say I want to consider this particular expression, $\sum_{i=1}^{n} \|x_i - \mu_{z_i}^{t+1}\|^2$. And then I want to compare it with a different quantity. And then we will talk about what these quantities are in a minute. So, this is $\sum_{i=1}^{n} \|x_i - \mu_{z_i}^{t}\|^2$. So, I have put down two quantities. Both of these are sum of distance squared of points to some other vectors, some other means.

But what is, let us look at this first. This is simpler thing to understand first. So, what is this quantity? Well, we are in the t-th iteration. Every point is being measured in this expression

to the mean of the box to which it is assigned to. You are computing the distance squared of every point to the mean of the box to which it is assigned to. So, x_i is assigned to the box Z_i^t.

That is by definition where x_i is assigned, Z_i^t is a cluster indicator of the ith data point in the t th iteration. So, the ith data point goes to the box Z_i^t, which is some number between 1 to k. So, basically, what this is capturing is just the sum of distances of each point to its own mean in the tth iteration. Now, what is this quantity on the left capturing? Well, this is doing something interesting.

So, it is trying to capture the distance of each point to the mean that it is being assigned to in the next round. So, let us be careful when I say mean, it is being assigned to the next round. Now, you are still looking at means of the tth round, you are in some partition there is μ_1^t, μ_2^t, μ_3^t, till μ_k^t, you are still measuring x_i to one of these, but which one, well, you are measuring it to the one which is in the box that you want to go to.

So, you are not measuring it every data points distance to its own mean, but then you are measuring its distance to the box where it wants to go. Well, some data points do not want to jump. So, it does not want to jump, the data point is happy with its own mean in which case Z_i^{t+1} for the data point will be same as Z_i^t. So, then it is the same mean that you are measuring it with.

But then there are some data points which might find that there is a mean which I am closer to so I want to jump and that is when the reassignment actually happens. Because the algorithm does not converge there is at least one point for which the distance to $\mu_{z_i^{t+1}}^t$. So, the mean of the box, where I want to go to is strictly lesser, the distance squared to that point mean is lesser than the distance squared to my own mean.

So, because there is at least one point with this property, and whenever a point decides to jump it is because this distance to where it is comparing itself to is strictly lesser than where it is currently assigned to. So, this whole sum is going to be less than the right-hand side. Well, this is simply by algorithm's choice. So, the algorithm makes this choice to make a jump only when it finds a different cluster where the mean is strictly closer to then the current cluster.

But remember, this quantity on the left is not the objective value of the partition that you would get in the t plus first round. Why? Because I have not made the jump it. So, this is saying I am closer to the other guy, and I am going to measure my distance to the other guy. So, that is this situation. So, whereas if you really want to measure the objective function in the next round, well, you should compare each point to the mean that you get in the next round, so after you have made the segments.

There are a lot of points which are jumping around, and after every point jumps around now you compute the mean in each box. And now each point is compared to its own mean, that is the objective function in the next round, but that is not what this is. So, this is an intermediate quantity if you will. So, let me write this. So, this is mean of cluster where x_i wants to go. And this is mean of current cluster where x_i is assigned to. That is the difference.

So, now what we really want to argue is how does the objective function change after you make the partition change. So, which means we want to see this quantity, $\sum_{i=1}^{n} \|x_i - \mu_{z_i^{t+1}}^{t+1}\|^2$, well in the t plus first round, how does Z_i^{t+1} th mean the one that x_i is being assigned to in the t plus first round, so we are comparing every data point to the mean in the t plus first round, and then we want to compare this quantities value.

Now, with the intermediate quantity. The intermediate quantity being $\sum_{i=1}^{n} \|x_i - \mu_{z_i^{t+1}}^{t}\|^2$. Now, what might be the relation between these two quantities. The quantity on the left is your partitions objective value evaluated in the t plus first round. The quantity on the right is an intermediate quantity, where after the t-th round, you are comparing each data point to the mean of the cluster where it wants to go.

So, now, how do these two values compare? What can we say about this quantity? Well, this would be a very instructive exercise to pause and think. I will tell you what the answer is and then I will argue why it is. The relation between these two terms is that the left-hand side term is less than or equal to the right-hand side. And why is this? Why should this be true? Now, instead of writing down a long argument as to why this is true, it is not really long argument, but then let us try to understand what is happening.

Let us focus on one box, one cluster. So, in the tth round, there were a bunch of points that were assigned to this cluster and this cluster had a specific mean. So, it had a specific mean. Now, there are some points which are happy with the mean, they are not changing. There are some points which are not happy with the mean, because they have found a different mean, which seems to be closer. So, those points are leaving this and then jumping out.

Now, there are some points which are remaining, but now there are some other points from different clusters, which think that this mean is closer and then they are coming here. So, now, if I look at the points of this particular cluster, after doing this reassignment, after the points, some points have gone out, some points have come in, now all those points are here. Why? Because they have been measured to the mean of the previous round, that is each x_i that is in cluster k. So, now, what does this compare it to on the right-hand side?

Well, this is comparing it to the mean of this box in the tth round. Now, what are we going to compare it against in the t plus first round, we are going to compare it against the mean of the t plus first round. So, after some points have gone out, some points have come in, now you have a set of points you compute the mean and then you are comparing the distance of each point to its own mean.

Earlier, the contribution to the right-hand side term for each point in this cluster was its distance to the previous mean of this cluster. Now, the contribution that each point to the objective function on the left-hand side is the distance to the mean of the updated cluster, so in the t plus first round. So, which means all the points in the right-hand side expression were contributing their distance to the mean of the tth round.

On the left-hand side expression, they are contributing the distance to the mean of the t plus first round. The question is which of the sum is smaller? Well, now this is where we use fact one. If you remember what we did, in fact one, we said that you have any bunch of points, does not matter what the set of points is, if you want to measure the distance or the contribution of each data point as a distance squared to a single point.

Well that contribution is smallest when the single point is the mean of the bunch of points. Which means in the t plus first round, because every point for many, if you break this expression into k different clusters, and then compare each cluster's points assigned to that cluster to its own mean, that would be smaller than the contribution of each of these points to

the previous mean, which is what these points were contributing in the right-hand side expression.

So, which means what is exactly happening is the following. So, this is less than or equal to this, well, because I can write this somehow like this, so I can write this expression as $\sum_{i=1}^{n}\sum_{k=1}^{K}\|x_i - \mu_R^{t+1}\|^2 \cdot \mathbb{1}\left(Z_i^{t+1} = k\right)$. Of course, I can write the left-hand side expression like this, because this is just, I am breaking the entire contribution of all the data points, which is what is summed here into each cluster.

So, now this is basically I can break this into a bunch of points that go into each cluster, and then see what is each points contribution to its own mean. Now, for every k, we know that, well, the set of points that belongs to that cluster, let us call that C_k. So, the data points distance to its own mean is less than or equal to the set of points, sum of distances to any v. So, this is fact 1, this was fact 1. The distance to its own mean, the sum of distance squares of data points to its own mean is less than or equal to sum of distance squared of point to any vector whatsoever.

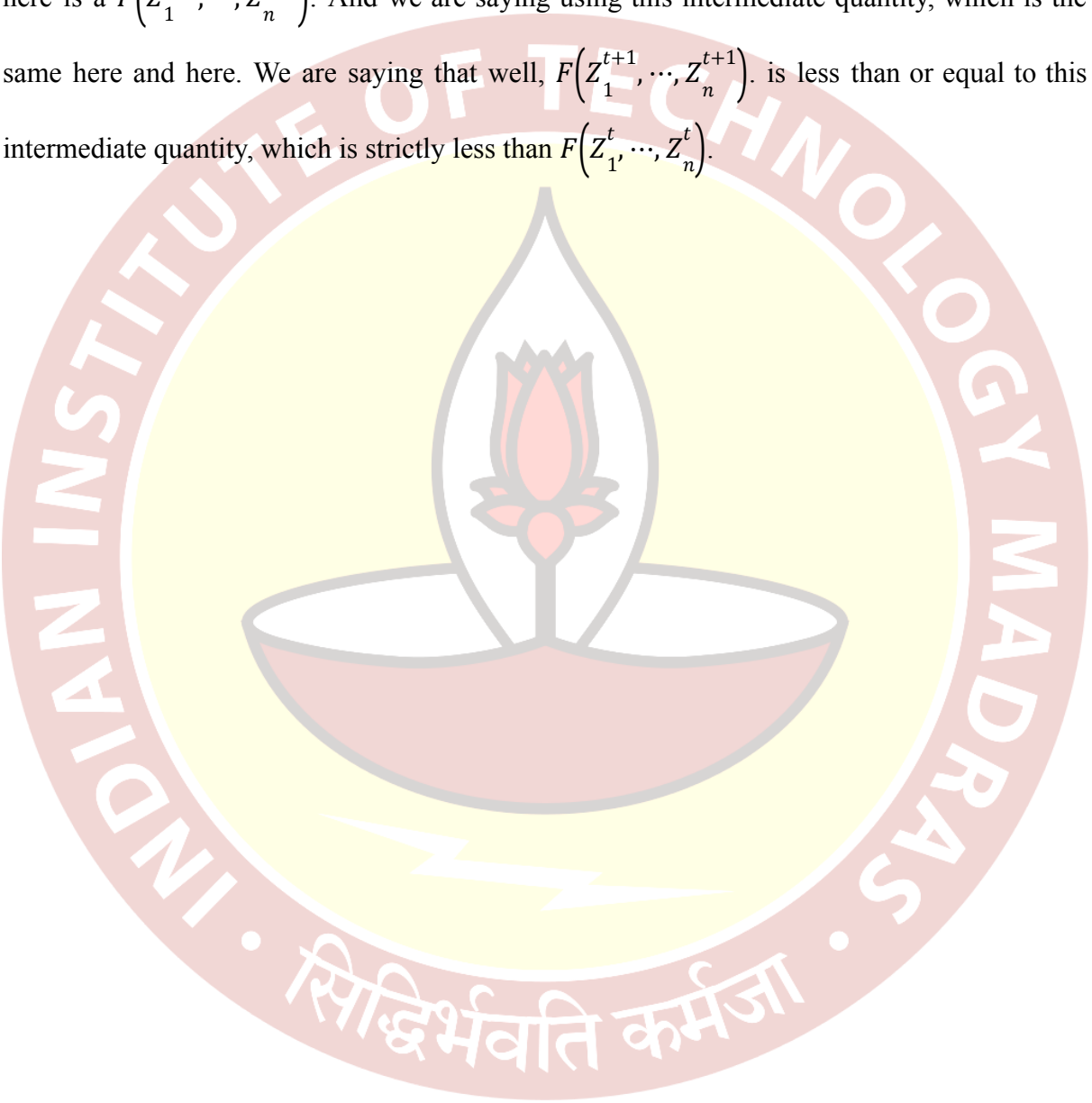
And in particular, this means that this has to be less than or equal to $\sum_{i \in C_k}\|x_i - \mu_{z_i^{t+1}}^{t}\|$. So, but then, so what is this expression? So, why am I saying I am justified in putting replacing this with v here, because all the points in cluster k were being compared, all the points in cluster k in the t plus first iteration were being compared to the mean of cluster k in the tth iteration. So, in fact, I can even say this is μ_k, that is okay, actually, μ_k.

So, this was why these points are in cluster k in the t plus first iteration. So, every point is in cluster k, because either it was compared to its own mean, and then it was happy or it compared itself to this mean, μ_k^t, and then jumped here. So, which means that, well, earlier it was being compared to μ_k^t, now it is being compared to μ_k^{t+1}. But because fact 1 says well μ_k^{t+1}'s comparison sum of distance squared should be less than the comparison to μ_k^t. So, which means this is true.

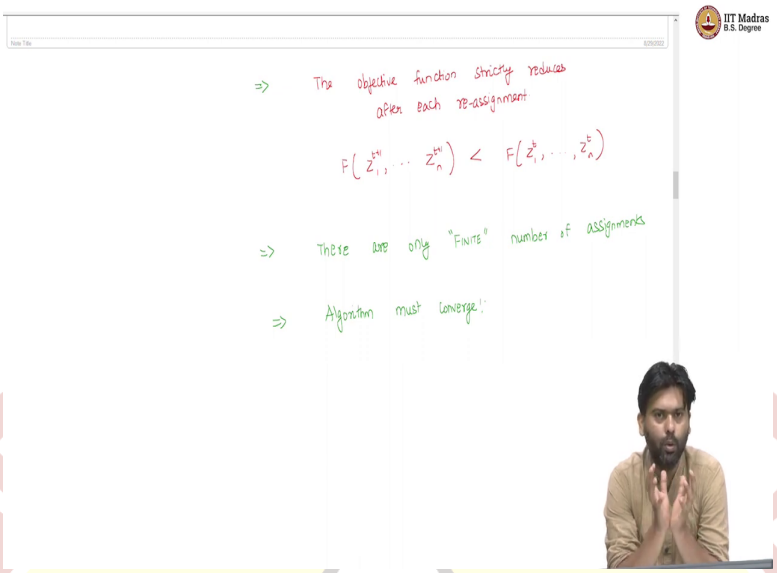
In other words, you can think as if points are moving closer and closer to each other in some sense. So, because after making this move your objective function reduces. That is what we

have kind of argued now. So, what is the argument? So, what have we gained by this? Well, we show that the objective function after making a reassignment strictly reduces, after every reassignment, that is what we have managed to show. How have we shown that?

Well, this quantity here is the objective function, this is just $F\left(Z_1^t, \cdots, Z_n^t\right)$. Now, this quantity here is a $F\left(Z_1^{t+1}, \cdots, Z_n^{t+1}\right)$. And we are saying using this intermediate quantity, which is the same here and here. We are saying that well, $F\left(Z_1^{t+1}, \cdots, Z_n^{t+1}\right)$. is less than or equal to this intermediate quantity, which is strictly less than $F\left(Z_1^t, \cdots, Z_n^t\right)$.



(Refer Slide Time: 26:36)



Which means what essentially then it says is that the objective function essentially what we are trying to argue is that the objective function strictly reduces after each reassignment that is $F\left(Z_1^{t+1}, \cdots, Z_n^{t+1}\right) < F\left(Z_1^t, \cdots, Z_n^t\right)$ if reassignment happens. Okay so, but why does this mean that the algorithm should converge? All we are saying is that we are in some partition, which has some objective value and now reassignment step happens and the objective value strictly reduces.

But why does strictly, say strict reduction in the objective value, imply that the algorithm should converge? Algorithms convergence remember means that you are reaching a specific partition where you know every point is happy with its own mean, no reassignment happens after that. That is what we mean by algorithm is converged. But why should this condition imply that the algorithm has converged?

This is a good question to pause and think. And I will answer that now. Well, in general, if an objective function keeps reducing, it could, one might imagine that well, it could keep on reducing without the algorithm converging. But now let us think about this way, what does it mean to say that the objective function strictly reduces, it means that the partition cannot repeat itself. You start with a partition it has a specific objective value.

Now, if a reassignment happens, the objective value strictly reduces, which means it has to be a different partition, it cannot be the same partition. So, the partition cannot repeat whenever reassignment happens. But there are only finite number of partitions. However large this

number could be, so, some huge number, the number of partitions of a bunch of n numbers into k boxes.

Naively, we saw that that grows exponentially, we will not really talk about the exact value of that, but then that is not the point. The point is that there are only a finite number of them, at most k^n of the m, but it is a huge number, still finite. Which means that every time a reassignment happens, I am thinking of crossing off one of these partitions.

So, because I have visited that partition, and because I cannot revisit a partition, eventually, your reassignments have to stop which means the algorithm necessarily has to converge. The finiteness of the number of partitions is what is allowing us to argue that this algorithm has to converge. There are only finite partitions, number of partitions assignments now that implies the algorithm must converge, that is the argument.

So, essentially, what we have managed to show is that every time a reassignment happens in the Lloyd's of the K-means algorithm, your objective function, which is measured as the sum of the distance of the data points to the mean of the clusters to which it is assigned distance squared, to which it is assigned, reduces monotonically, strictly monotonically.

And that implies that at some point, you will hit a partition where every point is happy with its own, and it will not change anymore, because it cannot infinitely keep reducing when you have only a finite bunch of possibilities. That is the argument. Now, it does not mean that, a couple of things, let me clarify that. It does not mean that the algorithm is always going to take k^n or the number of partition rounds to converge. This does not say that at all.

So, the finiteness argument does not imply that the algorithm necessarily takes those many rounds to converge. Nothing of that sort is being implied by this. The algorithm converges really fast. So, in practice, it typically does really fast for most practical datasets. This is the worst-case analysis. Even if it had to run for k^n, order of k^n iterations, well, it has to converge. That is the argument we are making.

We are not saying how many rounds the algorithm will take. That is point number 1. The second thing is that we are not really talking about the goodness of the partitions or the clustering that will result from this algorithm. Nothing of that sort is also being said here. So, we are not arguing that the algorithm will eventually because it converges, we are not saying that it has to converge to a place where the objective function is the smallest.

That is not true at all. All we are saying is that the algorithm will converge. But it could converge to some, in some sense, a local minima. So, at that partition everybody is happy. There is no more changes that the algorithm is trying to do. But that does not mean that that partition will give you the least objective value or all possible partitions. Nothing of that sort is also implied by this argument.

This argument just says the algorithm converges, which is a very important thing to know, because otherwise, we will not be sure. If partitions could repeat, then the algorithm can get in some loop and then it will never converge. We do not know when to stop. Now, that problem is not there is what this argument shows. So, that completes our argument about the convergence of the Lloyd's algorithm. We will next see the nature of clusters that the Lloyd's algorithm produces.